



VNiVERSiDAD  
D SALAMANCA

# Creating Electronic Books with Code and Artificial Intelligence Assistants

Course for the Faculties of Sciences and Chemical Sciences

Álvaro Lozano Murciego and André Sales Mendes

*Universidad de Salamanca*

1st Edition • 2026



# CONTENTS

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>Session 1. Introduction to Verilog</b>   | <b>3</b>  |
| 1.1      | Key concepts . . . . .                      | 3         |
| 1.2      | Guided practice . . . . .                   | 3         |
| 1.3      | Theory-practice-figures index . . . . .     | 4         |
| 1.4      | Reference figures . . . . .                 | 4         |
| 1.5      | Closing checkpoint . . . . .                | 5         |
| 1.6      | Original source . . . . .                   | 5         |
| <b>2</b> | <b>Session 2. Bit Operations</b>            | <b>7</b>  |
| 2.1      | Key concepts . . . . .                      | 7         |
| 2.2      | Guided practice . . . . .                   | 7         |
| 2.3      | Theory-practice-figures index . . . . .     | 8         |
| 2.4      | Work without an associated figure . . . . . | 8         |
| 2.5      | Closing checkpoint . . . . .                | 8         |
| 2.6      | Original source . . . . .                   | 8         |
| <b>3</b> | <b>Session 3. Logic Gates</b>               | <b>9</b>  |
| 3.1      | Key concepts . . . . .                      | 9         |
| 3.2      | Guided practice . . . . .                   | 9         |
| 3.3      | Theory-practice-figures index . . . . .     | 10        |
| 3.4      | Reference figures . . . . .                 | 10        |
| 3.5      | Closing checkpoint . . . . .                | 14        |
| 3.6      | Original source . . . . .                   | 14        |
| <b>4</b> | <b>Session 4. Modules</b>                   | <b>17</b> |
| 4.1      | Key concepts . . . . .                      | 17        |
| 4.2      | Guided practice . . . . .                   | 17        |
| 4.3      | Theory-practice-figures index . . . . .     | 18        |
| 4.4      | Reference figures . . . . .                 | 18        |
| 4.5      | Closing checkpoint . . . . .                | 21        |
| 4.6      | Original source . . . . .                   | 21        |
| <b>5</b> | <b>Session 5. Routers and Adders</b>        | <b>23</b> |
| 5.1      | Key concepts . . . . .                      | 23        |
| 5.2      | Guided practice . . . . .                   | 23        |
| 5.3      | Theory-practice-figures index . . . . .     | 24        |
| 5.4      | Reference figures . . . . .                 | 24        |
| 5.5      | Closing checkpoint . . . . .                | 28        |
| 5.6      | Original source . . . . .                   | 28        |
| <b>6</b> | <b>Session 6. Flip-Flops</b>                | <b>31</b> |

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| 6.1      | Key concepts . . . . .                  | 31        |
| 6.2      | Guided practice . . . . .               | 31        |
| 6.3      | Theory-practice-figures index . . . . . | 32        |
| 6.4      | Reference figures . . . . .             | 32        |
| 6.5      | Closing checkpoint . . . . .            | 35        |
| 6.6      | Original source . . . . .               | 35        |
| <b>7</b> | <b>Session 7. Registers</b>             | <b>37</b> |
| 7.1      | Key concepts . . . . .                  | 37        |
| 7.2      | Guided practice . . . . .               | 37        |
| 7.3      | Theory-practice-figures index . . . . . | 38        |
| 7.4      | Reference figures . . . . .             | 38        |
| 7.5      | Closing checkpoint . . . . .            | 40        |
| 7.6      | Original source . . . . .               | 40        |
| <b>8</b> | <b>Session 8. Counters</b>              | <b>41</b> |
| 8.1      | Key concepts . . . . .                  | 41        |
| 8.2      | Guided practice . . . . .               | 41        |
| 8.3      | Theory-practice-figures index . . . . . | 42        |
| 8.4      | Reference figures . . . . .             | 42        |
| 8.5      | Closing checkpoint . . . . .            | 47        |
| 8.6      | Original source . . . . .               | 47        |
| <b>9</b> | <b>Session 9. ALU</b>                   | <b>49</b> |
| 9.1      | Key concepts . . . . .                  | 49        |
| 9.2      | Guided practice . . . . .               | 49        |
| 9.3      | Theory-practice-figures index . . . . . | 50        |
| 9.4      | Reference figures . . . . .             | 50        |
| 9.5      | Closing checkpoint . . . . .            | 52        |
| 9.6      | Original source . . . . .               | 52        |

Welcome to the **Computers I** practice book.

This material supports the study of digital design fundamentals using **Verilog** as a hardware description language. The course starts from a simple idea: a computer is not a magic box, but an ordered composition of signals, logic gates, modules, registers, counters, and arithmetic-logic units.

## What you will learn

By working through this book, you will learn to:

- describe digital circuits with Verilog modules;
- simulate the behavior of logic gates, buses, flip-flops, registers, and counters;
- interpret signals, states, and propagation delays;
- build combinational and sequential components step by step;
- connect each exercise with the reference diagrams and tables for the session.

## How the course is organized

The course is organized into practice sessions. Each session includes a short theoretical explanation, guided exercises, an index connecting theory, practice, and figures, and reference diagrams to keep the circuit being programmed clearly in view.

### Recommended workflow

Before writing code, identify the circuit inputs, outputs, and internal signals. Then consult the reference figures, write the Verilog module, and check the simulation. If the result does not match what you expected, return to the figure and review the connections.

## Session roadmap

**Table1:** Initial map of the course

| Session | Topic                   | Main idea                                                       |
|---------|-------------------------|-----------------------------------------------------------------|
| 1       | Introduction to Verilog | Working environment, basic modules, and screen output           |
| 2       | Bit operations          | Masks, parity, and bit reduction                                |
| 3       | Logic gates             | AND, OR, NOT, NAND, NOR, XOR, XNOR, and combinational functions |
| 4       | Modules                 | Interfaces, hierarchy, comparators, and encoders                |
| 5       | Routers and adders      | Buses, multiplexers, half adders, and full adders               |
| 6       | Flip-flops              | Elementary memory, clock, edges, and asynchronous inputs        |
| 7       | Registers               | SISO/SIPO registers and waveform-based debugging                |
| 8       | Counters                | Up/down counting and state transitions                          |
| 9       | ALU                     | File inclusion and use of the 74181 ALU                         |

## Available material

Start with the first course session:

- *Session 1. Introduction to Verilog*

You can switch to the Spanish book from the language selector in the top bar.

## SESSION 1. INTRODUCTION TO VERILOG

This session introduces Verilog as a hardware description language and sets up the minimum workflow for compiling and simulating simple examples. The goal is not to write conventional software, but to describe circuits and observe their behavior.

### Learning objectives

- Verilog is an HDL: it describes hardware, not just a sequence of software instructions.
- The basic workflow is to write a `.v` file, compile it, and run the simulation.
- Modules delimit the design; an `initial` block defines a sequence of test actions.

## 1.1 Key concepts

- Verilog is an HDL: it describes hardware, not just a sequence of software instructions.
- The basic workflow is to write a `.v` file, compile it, and run the simulation.
- Modules delimit the design; an `initial` block defines a sequence of test actions.
- Registers (`reg`) store values during simulation and wires (`wire`) connect signals.
- The `$display` format codes help inspect values in binary, octal, decimal, or hexadecimal.

## 1.2 Guided practice

1. Create a working directory and save a `hello.v` file inside it.
2. Compile the file with `iverilog` and run the generated output.
3. Modify the example so that it prints an integer in decimal, binary, and hexadecimal; compare your output with [Fig. 1.1](#).
4. Declare a 16-bit register and observe what happens when assigning values beyond its capacity; use [Fig. 1.2](#) to identify which code fragment produces each output.

## 1.3 Theory-practice-figures index

Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

| Theory block                       | Related exercises | Figures to consult                                  |
|------------------------------------|-------------------|-----------------------------------------------------|
| Output formats and base conversion | Exercises 3 and 4 | <a href="#">Fig. 1.1</a> , <a href="#">Fig. 1.2</a> |

## 1.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The [Fig. 1.1](#) is the reference for: expected output for an introductory conversion and display exercise.

```
module ej_1_9;

integer i;
real f;

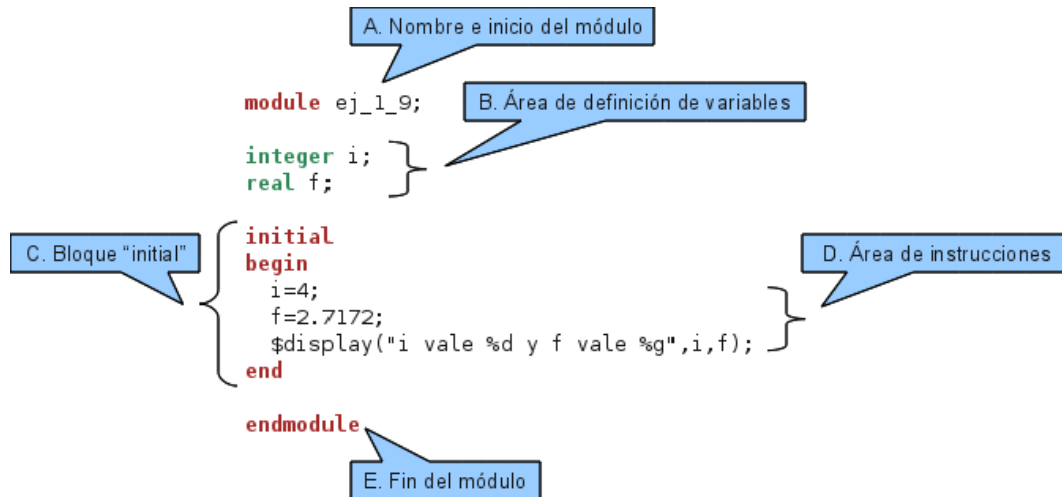
initial
begin
    i=4;
    f=2.7172;
    $display("i vale %d y f vale %g",i,f);
end

endmodule
```

**Figure1.1:** Expected output for an introductory conversion and display exercise.

The [Fig. 1.2](#) is the reference for: commented version of the same example, useful for locating each instruction in the code.





**Figure1.2:** Commented version of the same example, useful for locating each instruction in the code.

## 1.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

## 1.6 Original source

Content translated and expanded from the class presentation and the reference page: <http://avellano.fis.usal.es/~compi/sesion1.htm>.



## SESSION 2. BIT OPERATIONS

This session focuses on bit manipulation with masks. The central idea is to change selected positions of a register while leaving the rest unchanged.

### Learning objectives

- One's complement is obtained with  $\sim$ , turning each 0 into 1 and each 1 into 0.
- To set bits, use a mask with ones in the target positions and a bitwise OR operation.
- To clear bits, combine an inverted mask with a bitwise AND operation.

## 2.1 Key concepts

- One's complement is obtained with  $\sim$ , turning each 0 into 1 and each 1 into 0.
- To set bits, use a mask with ones in the target positions and a bitwise OR operation.
- To clear bits, combine an inverted mask with a bitwise AND operation.
- To toggle bits, use bitwise XOR with a mask marking the positions that must change.
- Reduction operators condense many bits into one result, for example to compute parity.

## 2.2 Guided practice

1. Declare a 16-bit register and assign an initial value that is easy to read in binary.
2. Set one bit, clear three bits, and toggle another bit using masks.
3. Compute the even parity bit of an 8-bit register.
4. Solve the operations by hand first and then compare them with the simulation; in this session the main reference is the bit table you build during the calculation.

## 2.3 Theory-practice-figures index

Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

| Theory block                     | Related exercises | Figures to consult                                                                      |
|----------------------------------|-------------------|-----------------------------------------------------------------------------------------|
| Masks, parity, and bit reduction | Exercises 1 to 4  | Manual bit table from the statement; no external figure is associated with this session |

## 2.4 Work without an associated figure

This session relies mainly on operations over registers and masks. Use a handwritten bit table to track each position before running the code.

## 2.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

## 2.6 Original source

Content translated and expanded from the class presentation and the reference page: <http://avellano.fis.usal.es/~compi/sesion2.htm>.

## SESSION 3. LOGIC GATES

This session connects the logic gates studied in theory with their instantiation in Verilog. It also introduces the special values  $x$  and  $z$ , which are essential in digital simulation.

### Learning objectives

- Verilog primitive gates are instantiated by listing the output first and then the inputs.
- The values  $x$  and  $z$  represent uncertainty and high impedance.
- A complete truth table must include 0, 1,  $x$ , and  $z$  when the exercise requires it.

### 3.1 Key concepts

- Verilog primitive gates are instantiated by listing the output first and then the inputs.
- The values  $x$  and  $z$  represent uncertainty and high impedance.
- A complete truth table must include 0, 1,  $x$ , and  $z$  when the exercise requires it.
- Gate interconnection makes it possible to build more complex combinational functions.
- The same function can be implemented in different ways, for example using only NAND gates.

### 3.2 Guided practice

1. Complete the AND truth table with the 16 combinations of 0, 1,  $x$ , and  $z$ ; use [Fig. 3.1](#) and [Fig. 3.2](#) as references.
2. Repeat the check for OR, NAND, NOR, XOR, XNOR, and BUFFER; consult [Fig. 3.3](#) through [Fig. 3.9](#) depending on the gate being simulated.
3. Build the proposed function with gates and verify its truth table; use [Fig. 3.10](#), [Fig. 3.11](#), and [Fig. 3.12](#).
4. Reimplement the function using only NAND gates and compare the outputs; follow [Fig. 3.14](#), [Fig. 3.13](#), and [Fig. 3.15](#).

### 3.3 Theory-practice-figures index

Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

| Theory block              | Related exercises | Figures to consult                                                                       |
|---------------------------|-------------------|------------------------------------------------------------------------------------------|
| Basic gates               | Exercises 1 and 2 | Fig. 3.1, Fig. 3.2, Fig. 3.3, Fig. 3.4, Fig. 3.5, Fig. 3.6, Fig. 3.7, Fig. 3.8, Fig. 3.9 |
| Combinational function f2 | Exercise 3        | Fig. 3.10, Fig. 3.11, Fig. 3.12                                                          |
| NAND equivalence          | Exercise 4        | Fig. 3.13, Fig. 3.14, Fig. 3.15                                                          |

### 3.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The Fig. 3.1 is the reference for: basic AND gate.

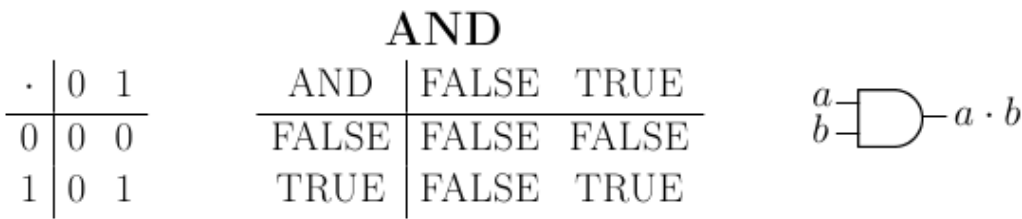


Figure3.1: Basic AND gate.

The Fig. 3.2 is the reference for: instantiation of an AND gate in Verilog.

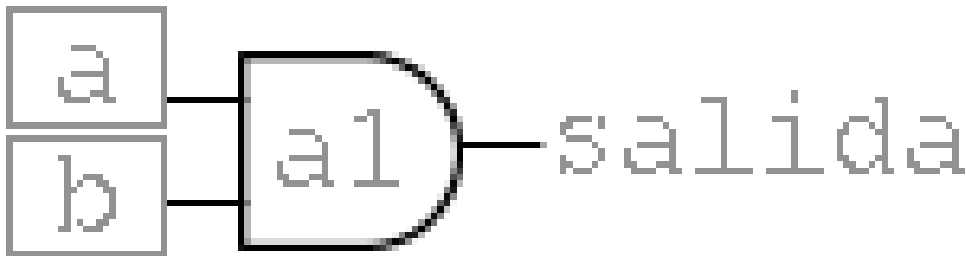


Figure3.2: Instantiation of an AND gate in Verilog.

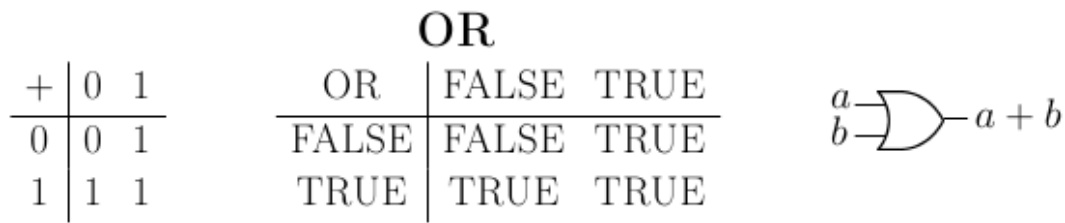
The Fig. 3.3 is the reference for: oR gate and its conceptual connection.

The Fig. 3.4 is the reference for: nOT gate.

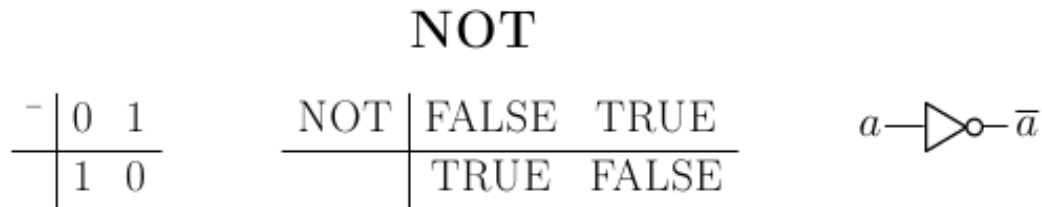
The Fig. 3.5 is the reference for: nAND gate.

The Fig. 3.6 is the reference for: nOR gate.

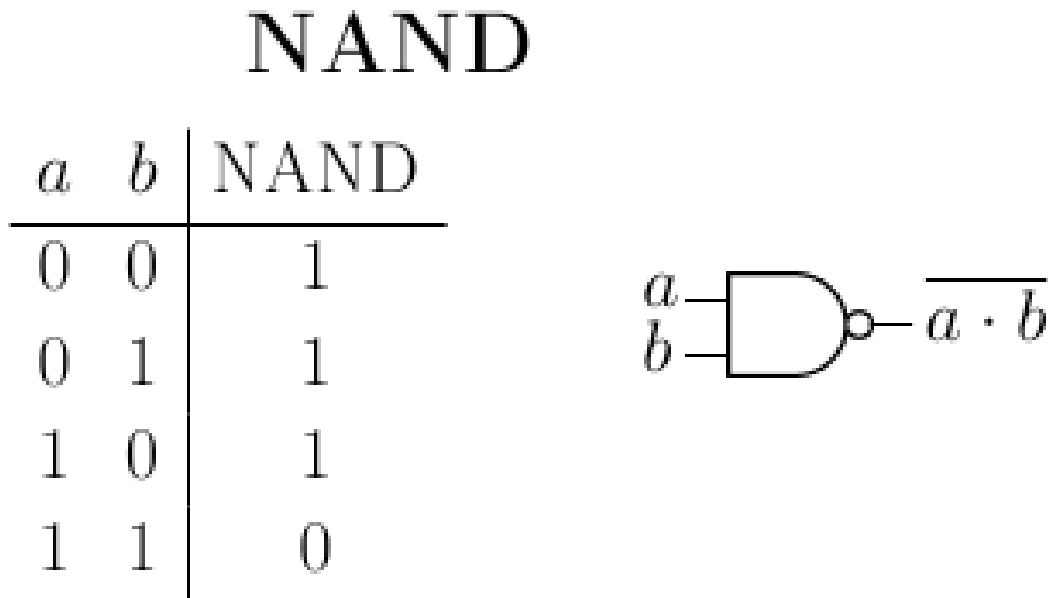
The Fig. 3.7 is the reference for: xOR gate.



**Figure3.3:** OR gate and its conceptual connection.



**Figure3.4:** NOT gate.



**Figure3.5:** NAND gate.

# NOR

| $a$ | $b$ | NOR |
|-----|-----|-----|
| 0   | 0   | 1   |
| 0   | 1   | 0   |
| 1   | 0   | 0   |
| 1   | 1   | 0   |



Figure3.6: NOR gate.

# XOR

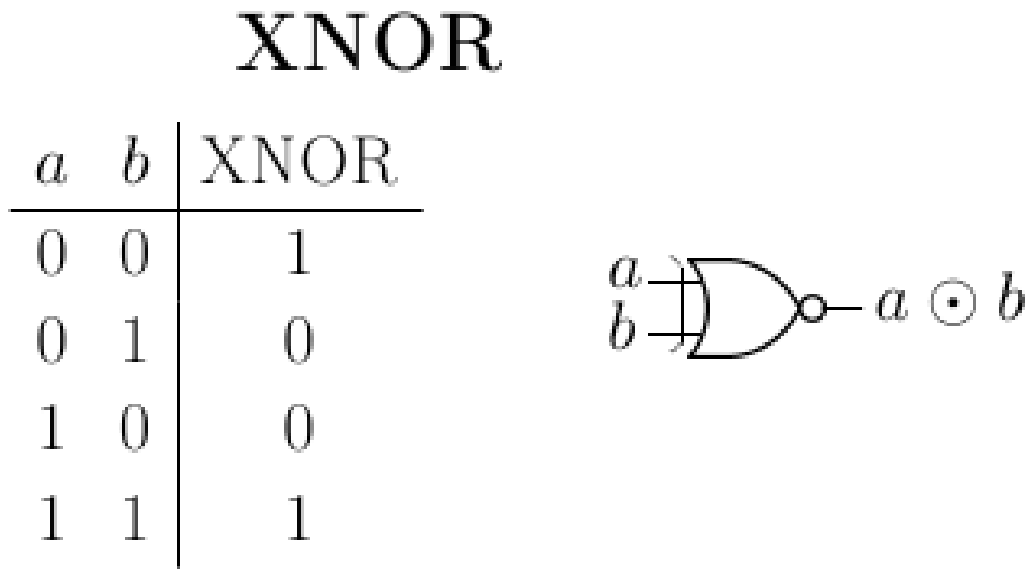
| $a$ | $b$ | XOR |
|-----|-----|-----|
| 0   | 0   | 0   |
| 0   | 1   | 1   |
| 1   | 0   | 1   |
| 1   | 1   | 0   |



Figure3.7: XOR gate.

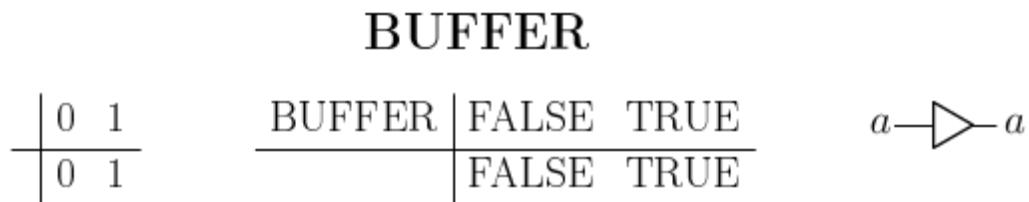


The Fig. 3.8 is the reference for: xNOR gate.



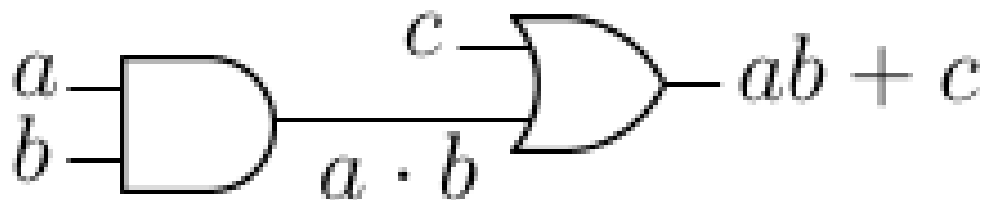
**Figure3.8:** XNOR gate.

The Fig. 3.9 is the reference for: bUFFER gate.



**Figure3.9:** BUFFER gate.

The Fig. 3.10 is the reference for: combinational logic function f2.



**Figure3.10:** Combinational logic function f2.

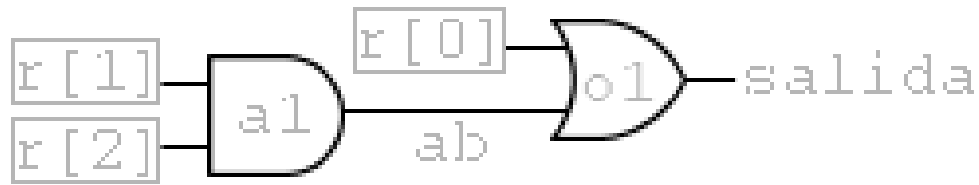
The Fig. 3.11 is the reference for: auxiliary variables for function f2.

The Fig. 3.12 is the reference for: truth table associated with f2.

The Fig. 3.13 is the reference for: algebraic form of function f3.

The Fig. 3.14 is the reference for: gate implementation of f3.

The Fig. 3.15 is the reference for: equivalent implementation using NAND gates.



**Figure3.11:** Auxiliary variables for function f2.

## 3.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

## 3.6 Original source

Content translated and expanded from the class presentation and the reference page: <http://avellano.fis.usal.es/~compi/sesion3.htm>.

| $a$ | $b$ | $c$ | $ab$ | $f_2$ |
|-----|-----|-----|------|-------|
| 0   | 0   | 0   | 0    | 0     |
| 0   | 0   | 1   | 0    | 1     |
| 0   | 1   | 0   | 0    | 0     |
| 0   | 1   | 1   | 0    | 1     |
| 1   | 0   | 0   | 0    | 0     |
| 1   | 0   | 1   | 0    | 1     |
| 1   | 1   | 0   | 1    | 1     |
| 1   | 1   | 1   | 1    | 1     |

Figure3.12: Truth table associated with  $f_2$ .

$$f_3(a, b, c) = bc + ab\bar{c} + \bar{b}c + c$$

Figure3.13: Algebraic form of function f3.

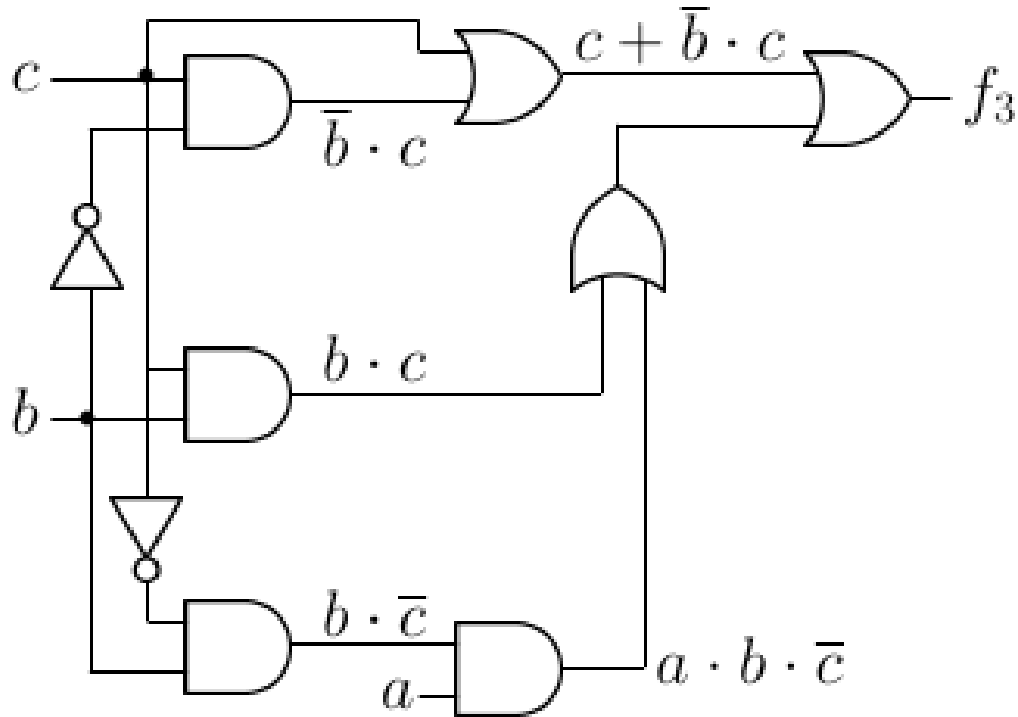


Figure3.14: Gate implementation of f3.

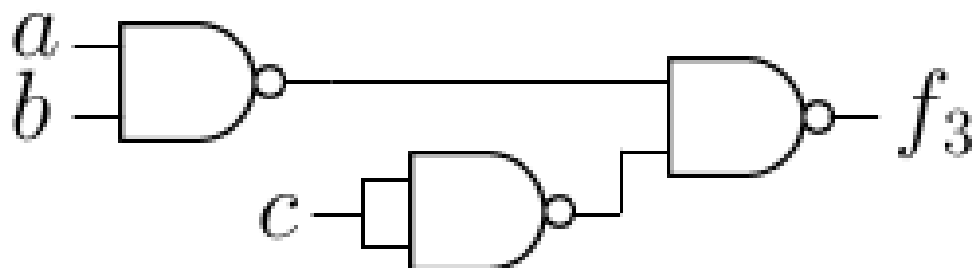


Figure3.15: Equivalent implementation using NAND gates.

## SESSION 4. MODULES

This session introduces modular design. A Verilog module encapsulates inputs, outputs, and behavior, like a reusable circuit block.

### Learning objectives

- A module needs a clear interface: `input`, `output`, and `inout`.
- Hierarchy lets us build large systems from smaller modules.
- Logic gates can be viewed as predefined modules in the language.

## 4.1 Key concepts

- A module needs a clear interface: `input`, `output`, and `inout`.
- Hierarchy lets us build large systems from smaller modules.
- Logic gates can be viewed as predefined modules in the language.
- A one-bit comparator is a good first example of a reusable module.
- Encoders show how a circuit changes when priority is introduced.

## 4.2 Guided practice

1. Define the interface of a one-bit comparator; identify inputs and outputs in [Fig. 4.2](#).
2. Implement the comparator with NOT, AND, and NOR gates; follow the internal structure in [Fig. 4.3](#).
3. Create a test module that enumerates all input combinations; compare your approach with [Fig. 4.4](#).
4. Use two one-bit comparators to build a two-bit comparator; take the hierarchy in [Fig. 4.5](#) as a reference.

## 4.3 Theory-practice-figures index

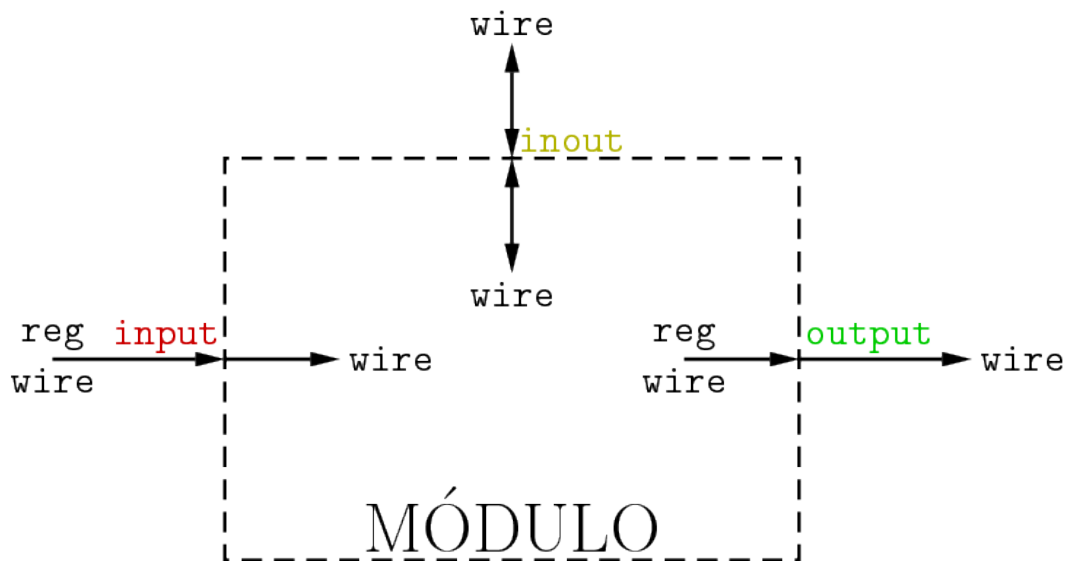
Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

| Theory block                | Related exercises         | Figures to consult           |
|-----------------------------|---------------------------|------------------------------|
| Module interface            | Exercise 1                | Fig. 4.1, Fig. 4.2           |
| Comparator internal circuit | Exercise 2                | Fig. 4.3                     |
| Test module                 | Exercise 3                | Fig. 4.4                     |
| Hierarchy and encoders      | Exercise 4 and extensions | Fig. 4.5, Fig. 4.6, Fig. 4.7 |

## 4.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The Fig. 4.1 is the reference for: general diagram of a module with ports.



**Figure4.1:** General diagram of a module with ports.

The Fig. 4.2 is the reference for: one-bit comparator.

The Fig. 4.3 is the reference for: internal implementation of the one-bit comparator.

The Fig. 4.4 is the reference for: comparator test module.

The Fig. 4.5 is the reference for: two-bit comparator built hierarchically.

The Fig. 4.6 is the reference for: priority encoder.

The Fig. 4.7 is the reference for: four-input AND gate.

| a | b | $a > b$ | $a = b$ | $a < b$ |
|---|---|---------|---------|---------|
| 0 | 0 | 0       | 1       | 0       |
| 0 | 1 | 0       | 0       | 1       |
| 1 | 0 | 1       | 0       | 0       |
| 1 | 1 | 0       | 1       | 0       |

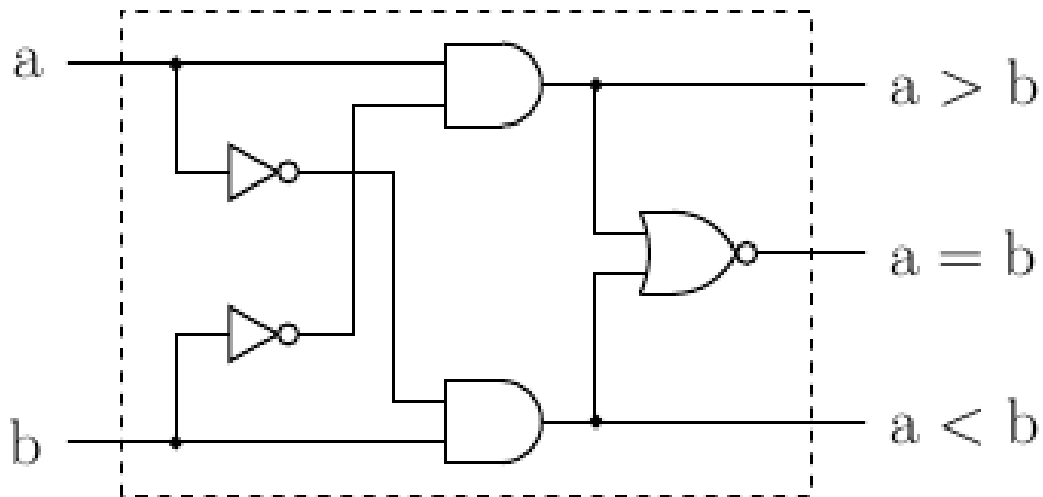


Figure4.2: One-bit comparator.

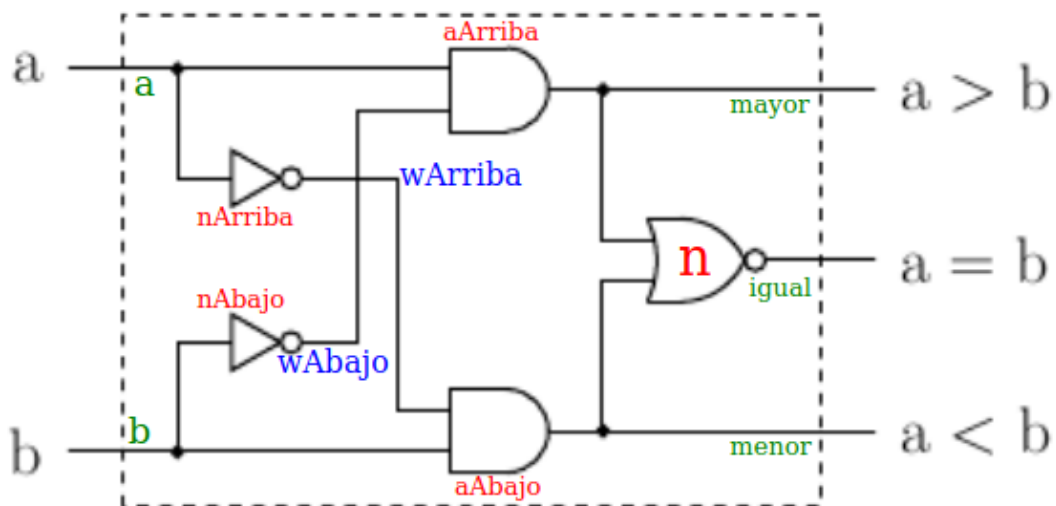


Figure4.3: Internal implementation of the one-bit comparator.

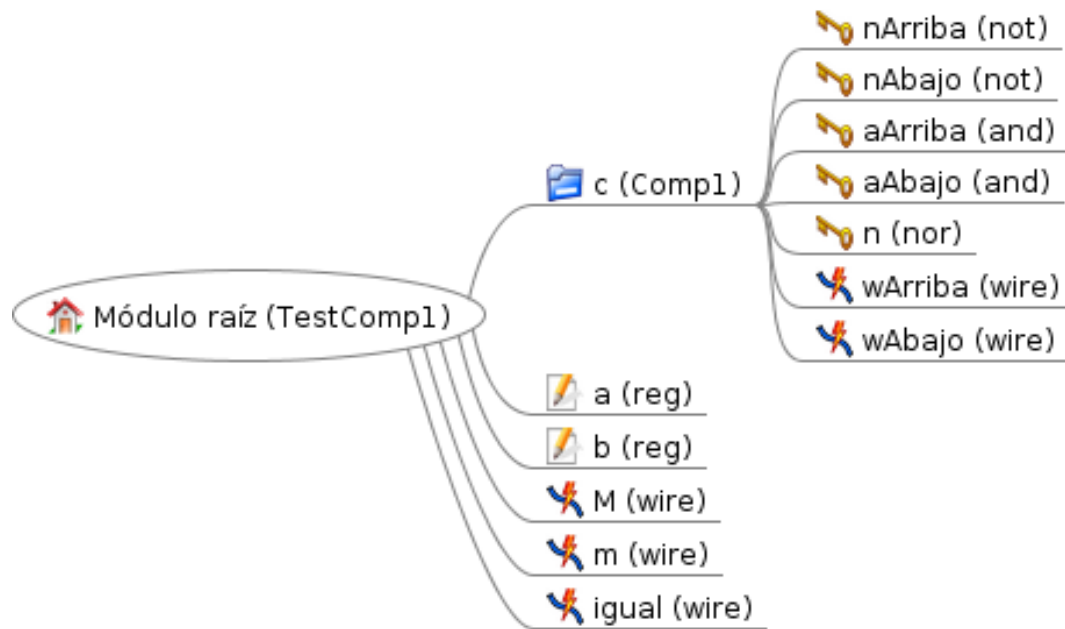


Figure4.4: Comparator test module.

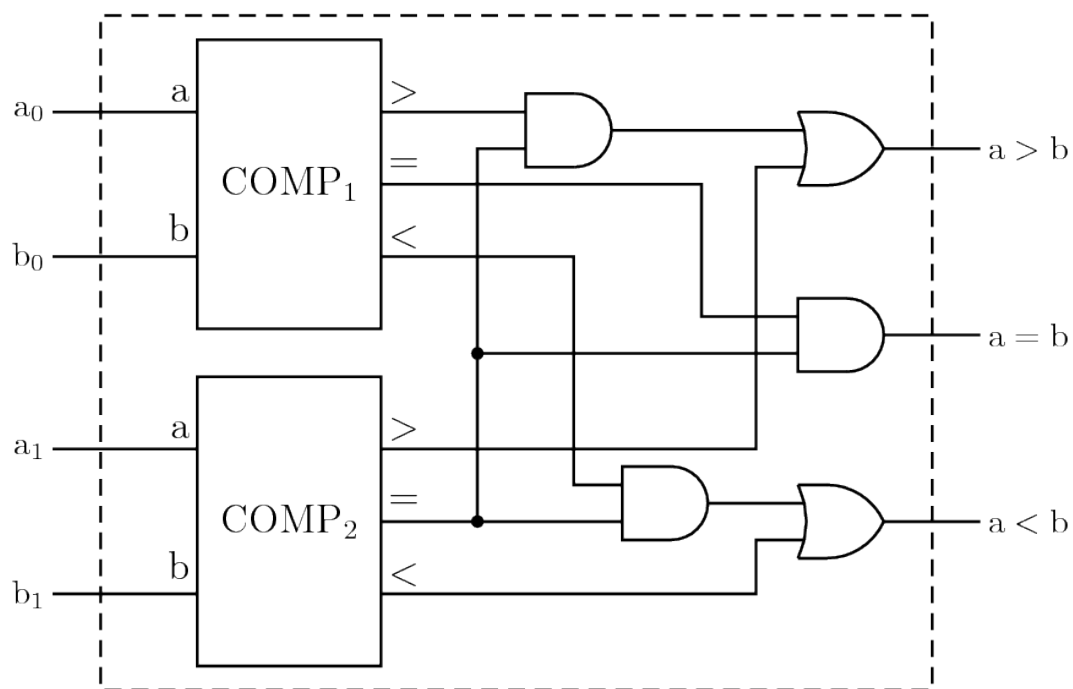
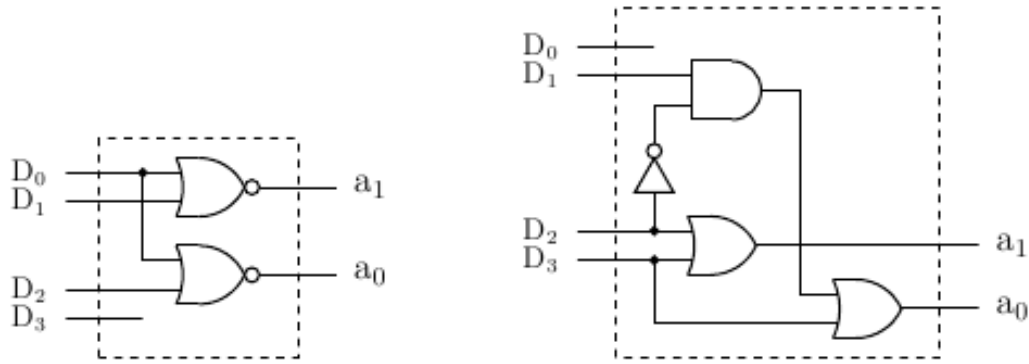
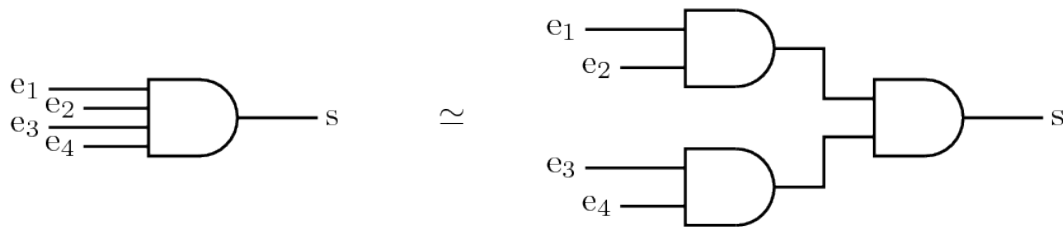


Figure4.5: Two-bit comparator built hierarchically.





**Figure4.6:** Priority encoder.



**Figure4.7:** Four-input AND gate.

## 4.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

## 4.6 Original source

Content translated and expanded from the class presentation and the reference page: <http://avellano.fis.usal.es/~compi/sesion4.htm>.



## SESSION 5. ROUTERS AND ADDERS

This session works with buses, tri-state buffers, multiplexers, and adders. The thread that connects them is controlling where a signal flows and how carry propagates.

### Learning objectives

- Tri-state buffers can place an output in high impedance.
- When several signals reach the same wire, types such as `tri`, `tri0`, `tri1`, `wand`, or `wor` are useful.
- `assign` creates a continuous connection between an expression and a wire.

### 5.1 Key concepts

- Tri-state buffers can place an output in high impedance.
- When several signals reach the same wire, types such as `tri`, `tri0`, `tri1`, `wand`, or `wor` are useful.
- `assign` creates a continuous connection between an expression and a wire.
- Multiplexers select one input among several using control lines.
- A full adder can be built from half adders and auxiliary gates.

### 5.2 Guided practice

1. Program a one-bit bus transceiver; use [Fig. 5.3](#) and [Fig. 5.4](#) to name the signals.
2. Build a 4x1 multiplexer with output enable and then an 8x1 multiplexer; use [Fig. 5.6](#) as a reference.
3. Implement a half adder and a full adder; relate your code to [Fig. 5.8](#) and [Fig. 5.9](#).
4. Add delays and estimate the safe stabilization time in a four-bit adder; justify the calculation with [Fig. 5.10](#) and compare it with [Fig. 5.11](#).

## 5.3 Theory-practice-figures index

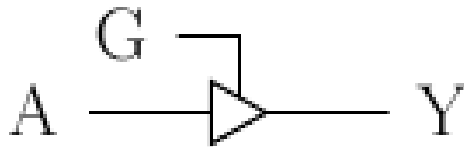
Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

| Theory block        | Related exercises | Figures to consult                                                                                                                   |
|---------------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Tri-state and buses | Exercise 1        | <a href="#">Fig. 5.1</a> , <a href="#">Fig. 5.2</a> , <a href="#">Fig. 5.3</a> , <a href="#">Fig. 5.4</a> , <a href="#">Fig. 5.5</a> |
| Multiplexers        | Exercise 2        | <a href="#">Fig. 5.6</a> , <a href="#">Fig. 5.7</a>                                                                                  |
| Adders              | Exercise 3        | <a href="#">Fig. 5.8</a> , <a href="#">Fig. 5.9</a>                                                                                  |
| Delays and carry    | Exercise 4        | <a href="#">Fig. 5.10</a> , <a href="#">Fig. 5.11</a>                                                                                |

## 5.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

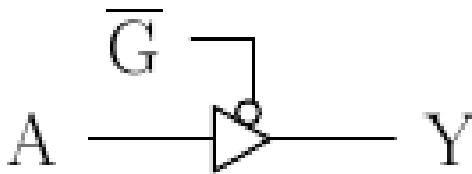
The [Fig. 5.1](#) is the reference for: tri-state buffer active high.



| G | A | Y |
|---|---|---|
| 0 | x | z |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**Figure5.1:** Tri-state buffer active high.

The [Fig. 5.2](#) is the reference for: tri-state buffer active low.



| $\overline{G}$ | A | Y |
|----------------|---|---|
| 0              | 0 | 0 |
| 0              | 1 | 1 |
| 1              | x | z |

**Figure5.2:** Tri-state buffer active low.

The [Fig. 5.3](#) is the reference for: one-bit bus transceiver.

The [Fig. 5.4](#) is the reference for: auxiliary signal names in the transceiver.

The [Fig. 5.5](#) is the reference for: transceiver test module.

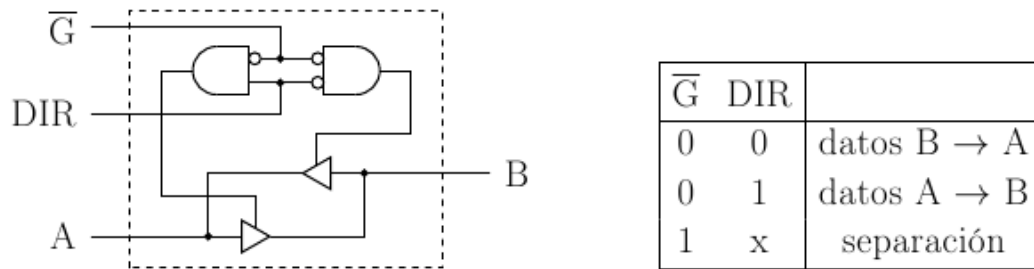


Figure5.3: One-bit bus transceiver.

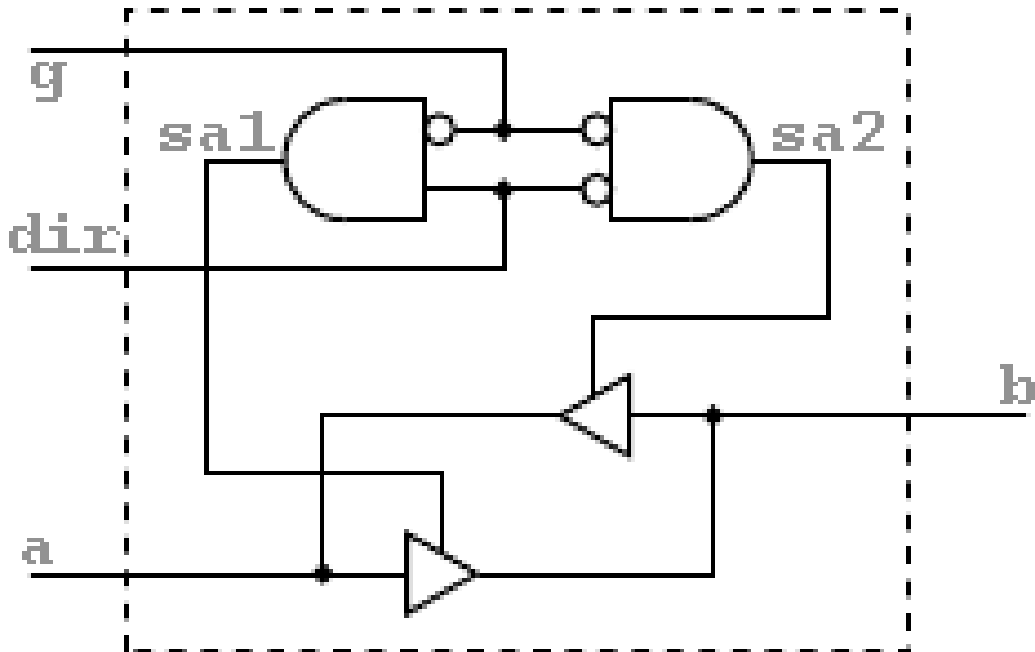


Figure5.4: Auxiliary signal names in the transceiver.

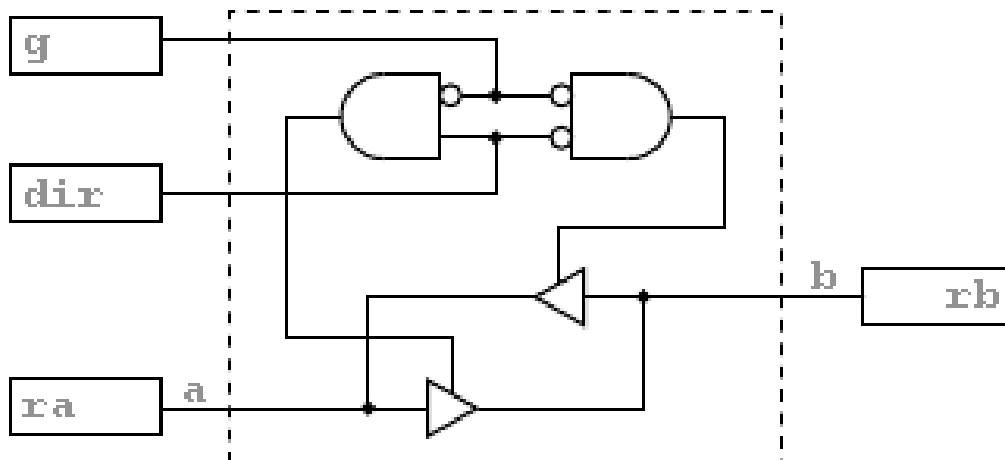
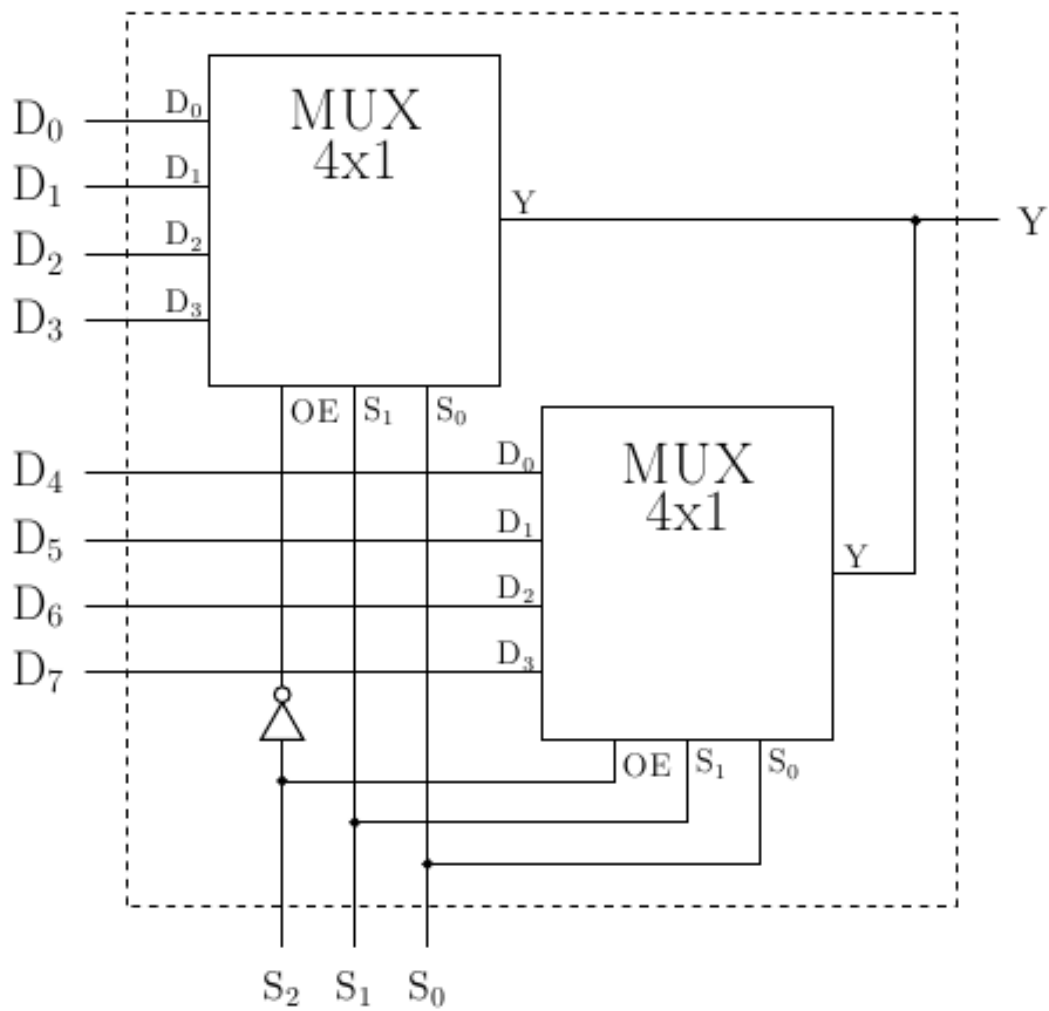


Figure5.5: Transceiver test module.



**Figure5.6:** 8x1 multiplexer built from smaller multiplexers.

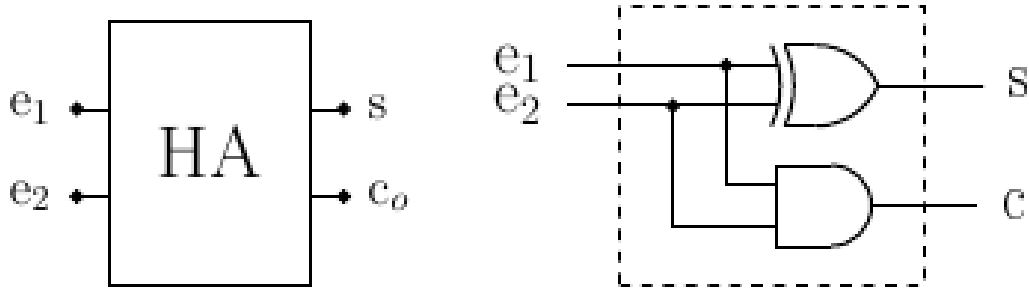
The Fig. 5.6 is the reference for: 8x1 multiplexer built from smaller multiplexers.

The Fig. 5.7 is the reference for: auxiliary structure for signal selection.

$$h(a, b, c, d) = \bar{a}\bar{b}\bar{d} + b\bar{c}d + ab + a\bar{c}$$

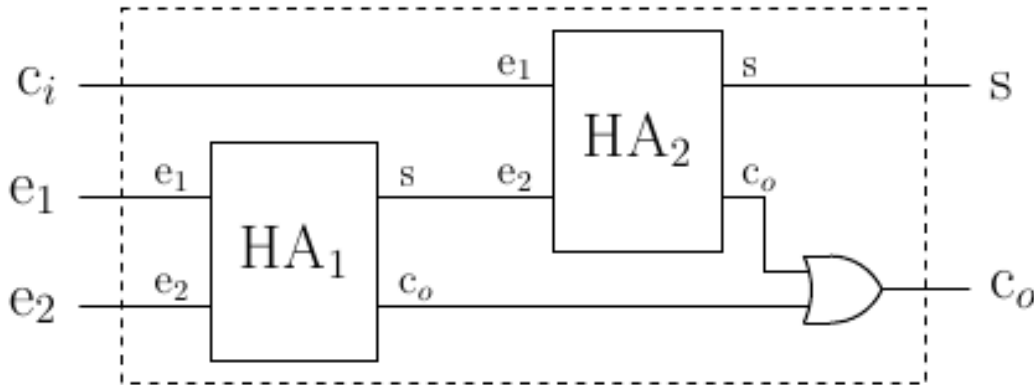
**Figure5.7:** Auxiliary structure for signal selection.

The Fig. 5.8 is the reference for: half adder.



**Figure5.8:** Half adder.

The Fig. 5.9 is the reference for: one-bit full adder.



**Figure5.9:** One-bit full adder.

The Fig. 5.10 is the reference for: four-bit ripple-carry adder.

The Fig. 5.11 is the reference for: carry-lookahead adder.

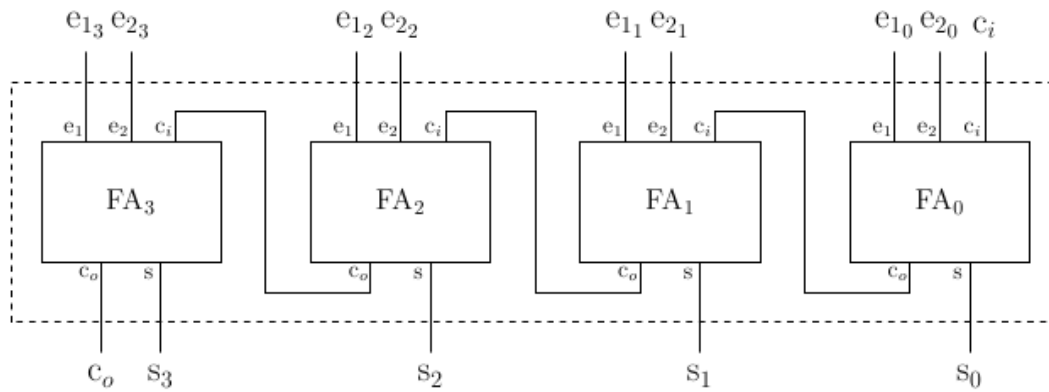


Figure5.10: Four-bit ripple-carry adder.

## 5.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

## 5.6 Original source

Content translated and expanded from the class presentation and the reference page: <http://avellano.fis.usal.es/~compi/session5.htm>.



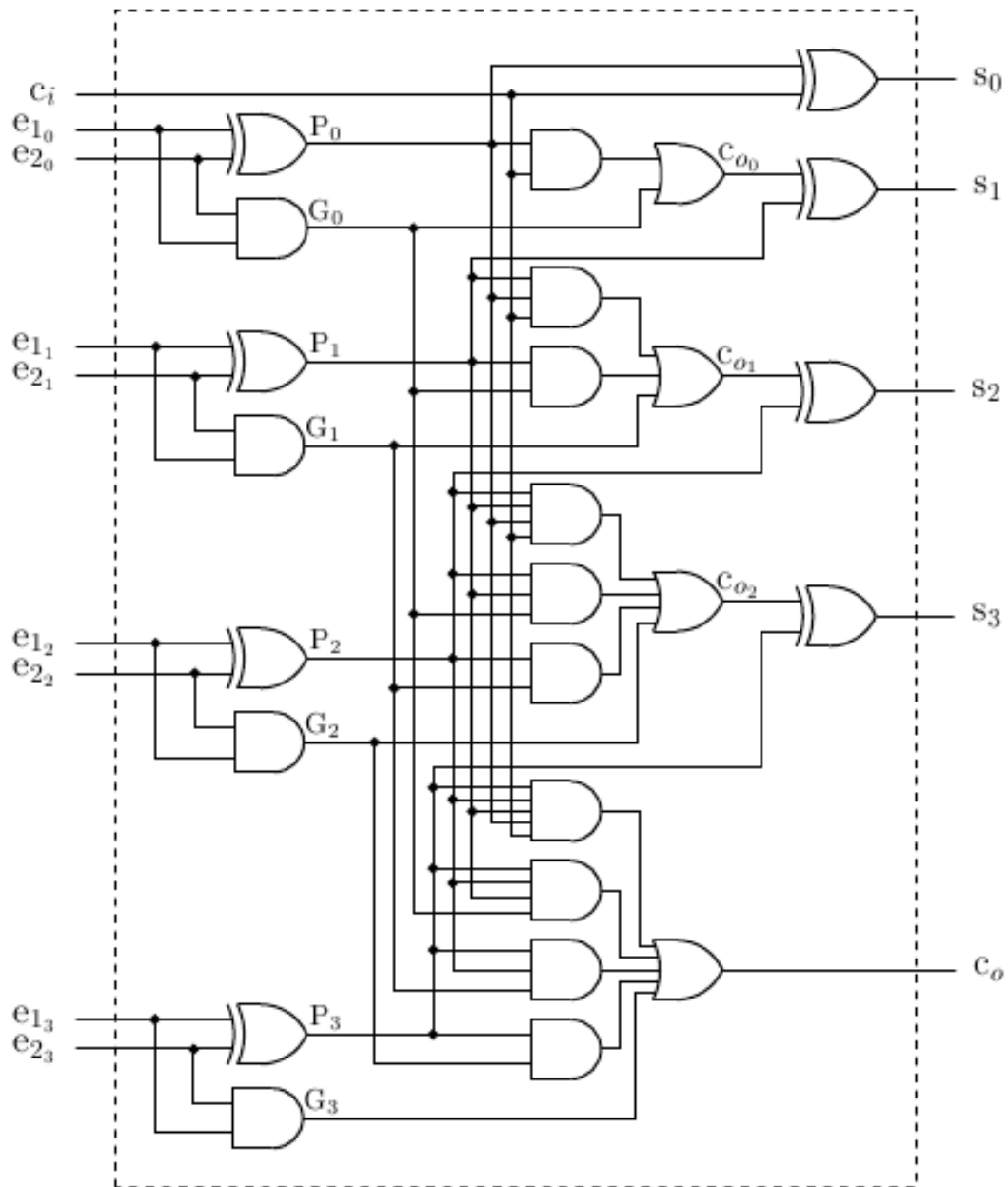


Figure5.11: Carry-lookahead adder.



## SESSION 6. FLIP-FLOPS

This session moves from combinational logic to memory. Flip-flops keep state, which forces us to reason about clocks, edges, asynchronous inputs, and delays.

### Learning objectives

- An RS flip-flop can be built with two cross-coupled NOR gates.
- Blocking and non-blocking assignments do not always have the same timing effect.
- A clock can be generated with an `always` block that periodically toggles a signal.

## 6.1 Key concepts

- An RS flip-flop can be built with two cross-coupled NOR gates.
- Blocking and non-blocking assignments do not always have the same timing effect.
- A clock can be generated with an `always` block that periodically toggles a signal.
- Edge detectors convert a level condition into a short pulse.
- PRESET and CLEAR are often asynchronous and take priority over the clock.

## 6.2 Guided practice

1. Program an RS flip-flop and run the proposed input sequence; consult [Fig. 6.1](#) and compare the results with [Fig. 6.2](#).
2. Replace some assignments with non-blocking assignments and observe the differences near the invalid states shown in [Fig. 6.2](#).
3. Add a level-sensitive clock input; use [Fig. 6.3](#) and [Fig. 6.4](#) to check when the output should change.
4. Build an edge-triggered version and then a JK flip-flop with PRESET and CLEAR; follow [Fig. 6.5](#), [Fig. 6.6](#), and [Fig. 6.7](#).

## 6.3 Theory-practice-figures index

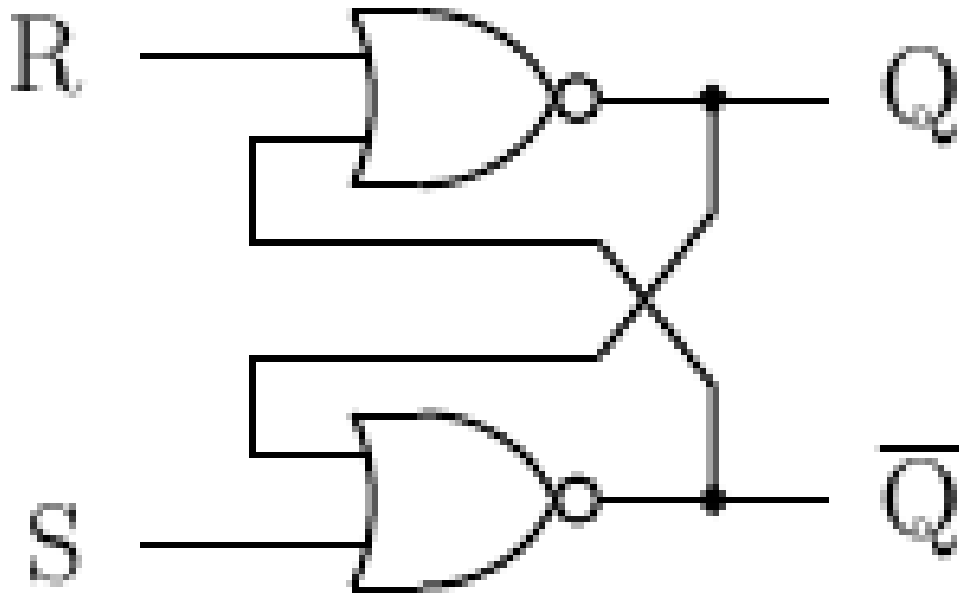
Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

| Theory block          | Related exercises | Figures to consult                                                             |
|-----------------------|-------------------|--------------------------------------------------------------------------------|
| RS flip-flop          | Exercises 1 and 2 | <a href="#">Fig. 6.1</a> , <a href="#">Fig. 6.2</a>                            |
| Level-sensitive clock | Exercise 3        | <a href="#">Fig. 6.3</a> , <a href="#">Fig. 6.4</a>                            |
| Edges and JK          | Exercise 4        | <a href="#">Fig. 6.5</a> , <a href="#">Fig. 6.6</a> , <a href="#">Fig. 6.7</a> |

## 6.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The [Fig. 6.1](#) is the reference for: rS flip-flop with NOR gates.



**Figure 6.1:** RS flip-flop with NOR gates.

The [Fig. 6.2](#) is the reference for: rS flip-flop behavior table.

The [Fig. 6.3](#) is the reference for: clocked RS flip-flop.

The [Fig. 6.4](#) is the reference for: clocked RS flip-flop behavior table.

The [Fig. 6.5](#) is the reference for: edge detector.

The [Fig. 6.6](#) is the reference for: jK flip-flop with control inputs.

The [Fig. 6.7](#) is the reference for: jK flip-flop behavior table.

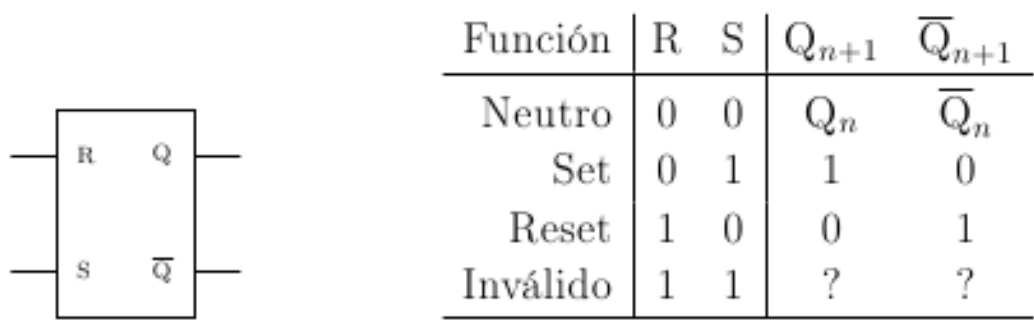


Figure6.2: RS flip-flop behavior table.

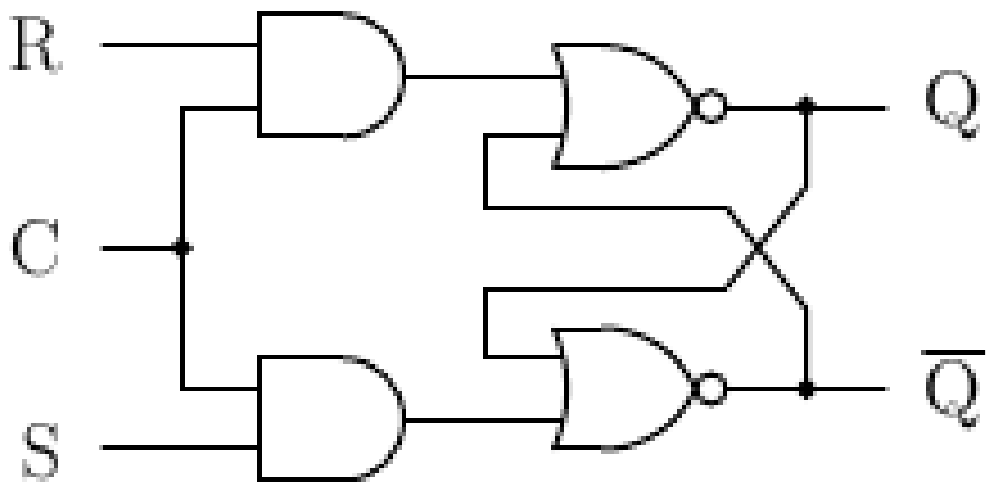


Figure6.3: Clocked RS flip-flop.

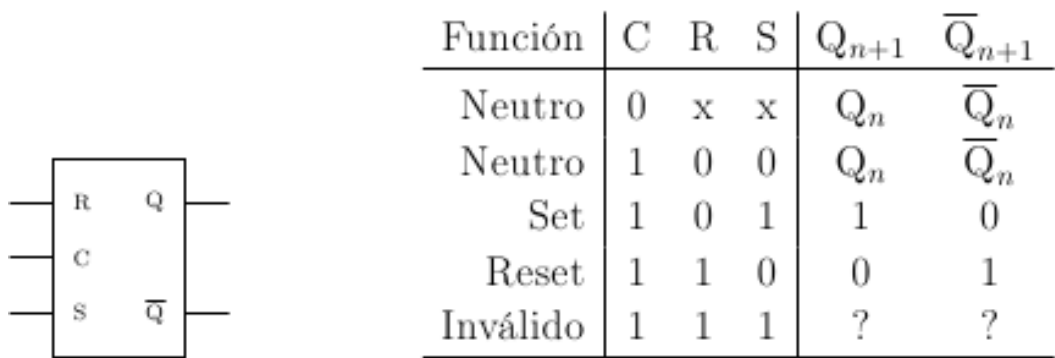


Figure6.4: Clocked RS flip-flop behavior table.

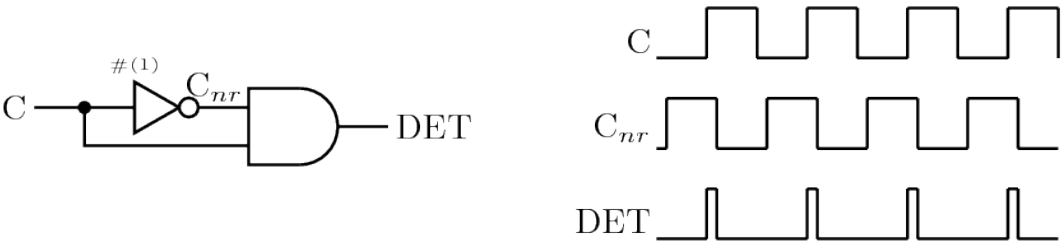


Figure6.5: Edge detector.

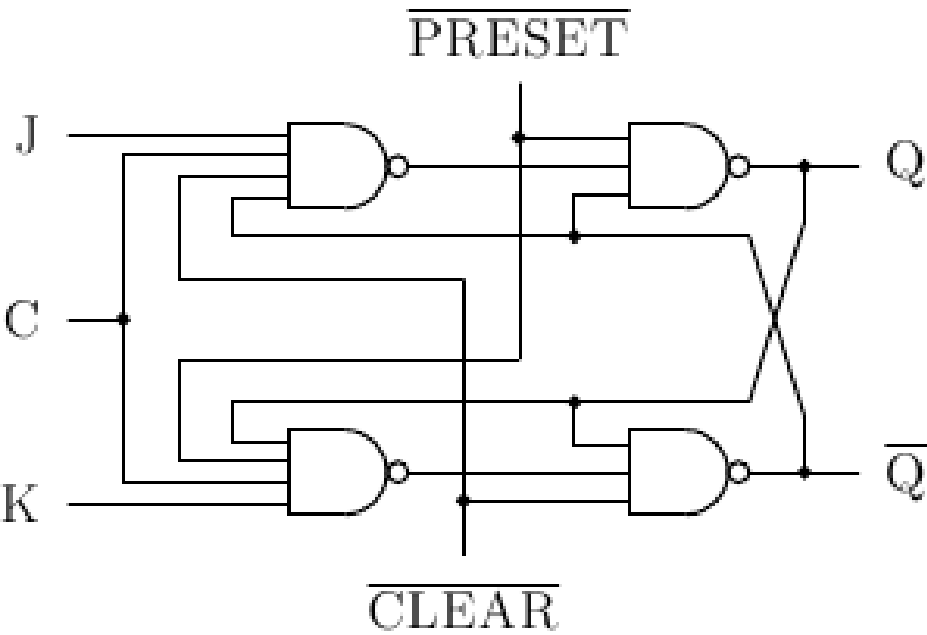


Figure6.6: JK flip-flop with control inputs.

|  | Función     | $\overline{PR}$ | $\overline{CL}$ | C | J | K | $Q_{n+1}$        | $\overline{Q}_{n+1}$ |
|--|-------------|-----------------|-----------------|---|---|---|------------------|----------------------|
|  | Preset      | 0               | 1               | x | x | x | 1                | 0                    |
|  | Clear       | 1               | 0               | x | x | x | 0                | 1                    |
|  | Inválido    | 0               | 0               | x | x | x | ?                | ?                    |
|  | Neutro      | 1               | 1               | 1 | 0 | 0 | $Q_n$            | $\overline{Q}_n$     |
|  | Set         | 1               | 1               | 1 | 1 | 0 | 1                | 0                    |
|  | Reset       | 1               | 1               | 1 | 0 | 1 | 0                | 1                    |
|  | Complemento | 1               | 1               | 1 | 1 | 1 | $\overline{Q}_n$ | $Q_n$                |
|  | Neutro      | 1               | 1               | 0 | x | x | $Q_n$            | $\overline{Q}_n$     |

Figure6.7: JK flip-flop behavior table.

## 6.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

## 6.6 Original source

Content translated and expanded from the class presentation and the reference page: <http://avellano.fis.usal.es/~compi/sesion6.htm>.





## SESSION 7. REGISTERS

This session builds registers from flip-flops and introduces an essential debugging tool: waveforms with GTKWave.

### Learning objectives

- A register stores several bits distributed across flip-flops.
- SISO registers shift information serial-in, serial-out.
- SIPO registers receive data serially and output it in parallel.

## 7.1 Key concepts

- A register stores several bits distributed across flip-flops.
- SISO registers shift information serial-in, serial-out.
- SIPO registers receive data serially and output it in parallel.
- `$dumpfile` and `$dumpvars` generate waveform files for signal inspection.
- Real or simulated delays can explain unexpected behavior.

## 7.2 Guided practice

1. Build a D flip-flop from the blocks already studied; use [Fig. 7.1](#) to verify inputs and output.
2. Use several flip-flops to form a SISO register; follow the connection in [Fig. 7.2](#).
3. Generate a waveform file and open it with GTKWave; compare the expected window with [Fig. 7.3](#).
4. Modify the register to obtain a parallel SIPO output; use [Fig. 7.4](#) and, for the extension, [Fig. 7.5](#).

### 7.3 Theory-practice-figures index

Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

| Theory block                     | Related exercises | Figures to consult |
|----------------------------------|-------------------|--------------------|
| D flip-flop                      | Exercise 1        | Fig. 7.1           |
| SISO register                    | Exercise 2        | Fig. 7.2           |
| Waveforms                        | Exercise 3        | Fig. 7.3           |
| SIPO registers and parallel load | Exercise 4        | Fig. 7.4, Fig. 7.5 |

### 7.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The Fig. 7.1 is the reference for: d flip-flop.

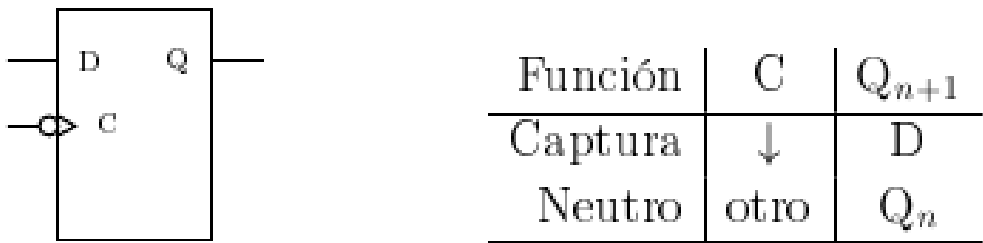


Figure7.1: D flip-flop.

The Fig. 7.2 is the reference for: sISO register.

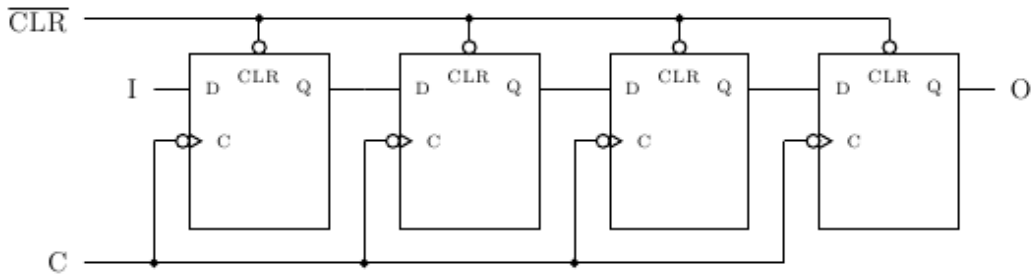


Figure7.2: SISO register.

The Fig. 7.3 is the reference for: signal visualization with GTKWave.

The Fig. 7.4 is the reference for: sIPO register.

The Fig. 7.5 is the reference for: register with parallel load and serial shift.

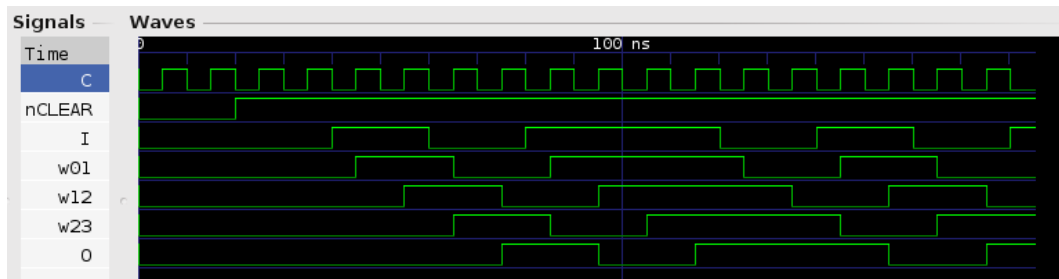


Figure7.3: Signal visualization with GTKWave.

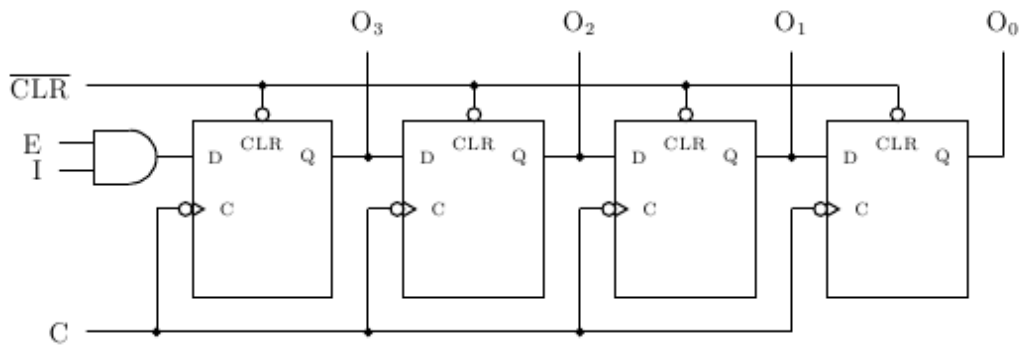


Figure7.4: SIPO register.

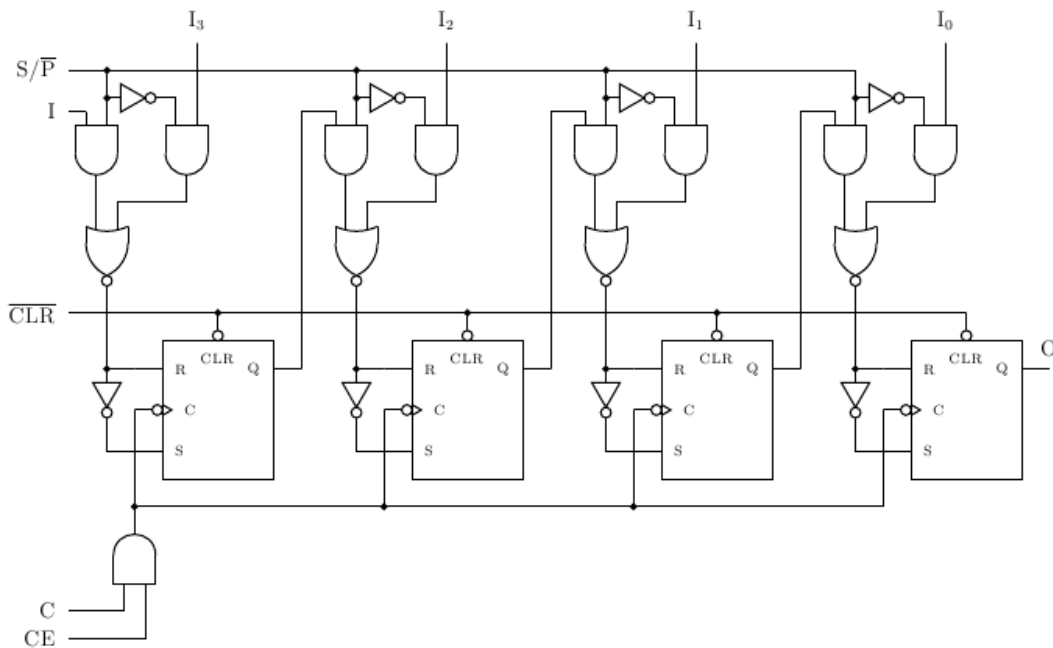


Figure7.5: Register with parallel load and serial shift.

## 7.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

## 7.6 Original source

Content translated and expanded from the class presentation and the reference page: <http://avellano.fis.usal.es/~compi/sesion7.htm>.

## SESSION 8. COUNTERS

This session uses flip-flops to build counters and analyze state transitions. The important point is to distinguish ideal behavior, delays, and transient states.

### Learning objectives

- A counter changes state according to a clock signal.
- Counting can be upward or downward depending on a mode line.
- HOLD freezes the state when the count direction must change without jumps.

## 8.1 Key concepts

- A counter changes state according to a clock signal.
- Counting can be upward or downward depending on a mode line.
- HOLD freezes the state when the count direction must change without jumps.
- Asynchronous counters can pass through transient states while signals propagate.
- State transition diagrams summarize the sequential behavior of a circuit.

## 8.2 Guided practice

1. Program a four-bit up/down counter; use [Fig. 8.1](#) to define inputs, outputs, and count mode.
2. Observe what happens when changing mode without clearing the counter; explain the transient states with [Fig. 8.4](#).
3. Add PRESET, CLEAR, and HOLD to control transitions better; compare your design with [Fig. 8.3](#) and [Fig. 8.5](#).
4. Analyze an arbitrary-count counter and draw its state diagram; use [Fig. 8.7](#), [Fig. 8.8](#), and [Fig. 8.9](#).

### 8.3 Theory-practice-figures index

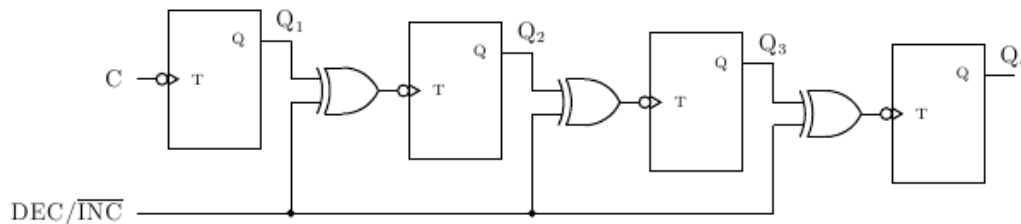
Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

| Theory block             | Related exercises | Figures to consult                     |
|--------------------------|-------------------|----------------------------------------|
| Up/down counting         | Exercises 1 and 2 | Fig. 8.1, Fig. 8.4                     |
| Transition control       | Exercise 3        | Fig. 8.2, Fig. 8.3, Fig. 8.5           |
| States and JK flip-flops | Exercise 4        | Fig. 8.6, Fig. 8.7, Fig. 8.8, Fig. 8.9 |

## 8.4 Reference figures

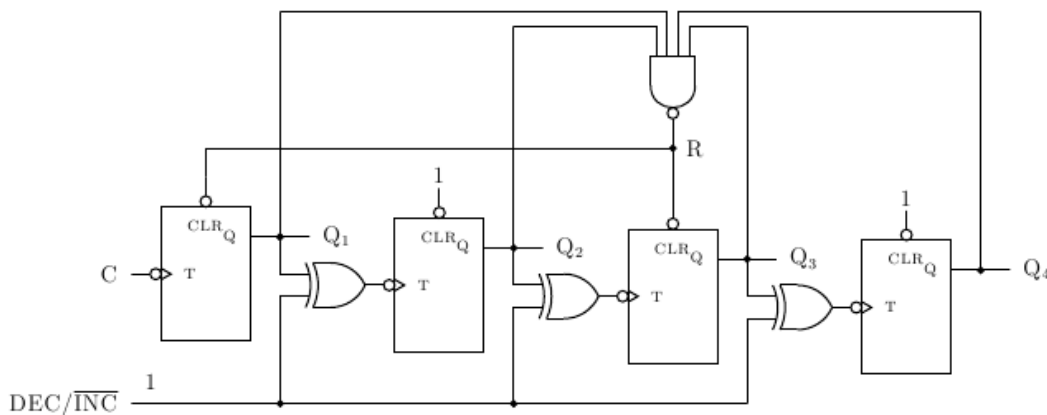
The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The Fig. 8.1 is the reference for: counter with up/down selection.



**Figure8.1:** Counter with up/down selection.

The Fig. 8.2 is the reference for: counter from 10 down to 0.



**Figure8.2:** Counter from 10 down to 0.

The Fig. 8.3 is the reference for: synchronous counter.

The Fig. 8.4 is the reference for: analysis of states and transitions.

The Fig. 8.5 is the reference for: auxiliary counter circuit.

The Fig. 8.6 is the reference for: transitions of a JK flip-flop.

The Fig. 8.7 is the reference for: state transition diagram.

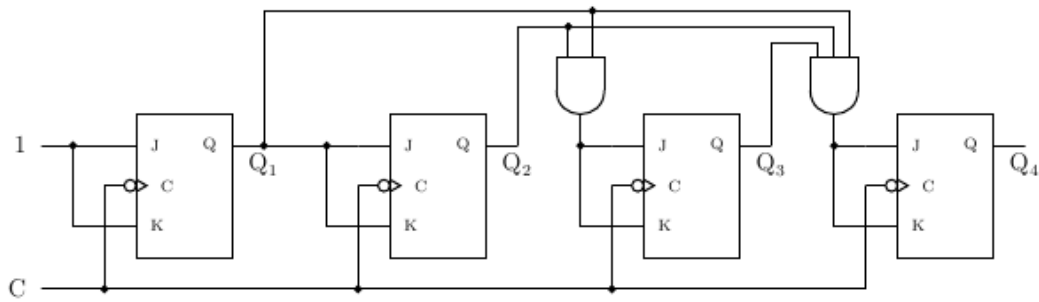


Figure8.3: Synchronous counter.

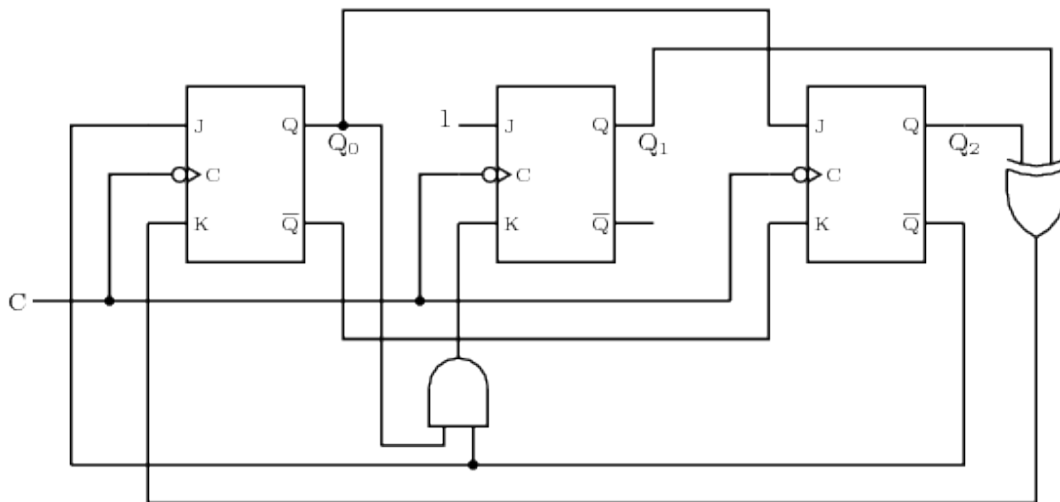


Figure8.4: Analysis of states and transitions.

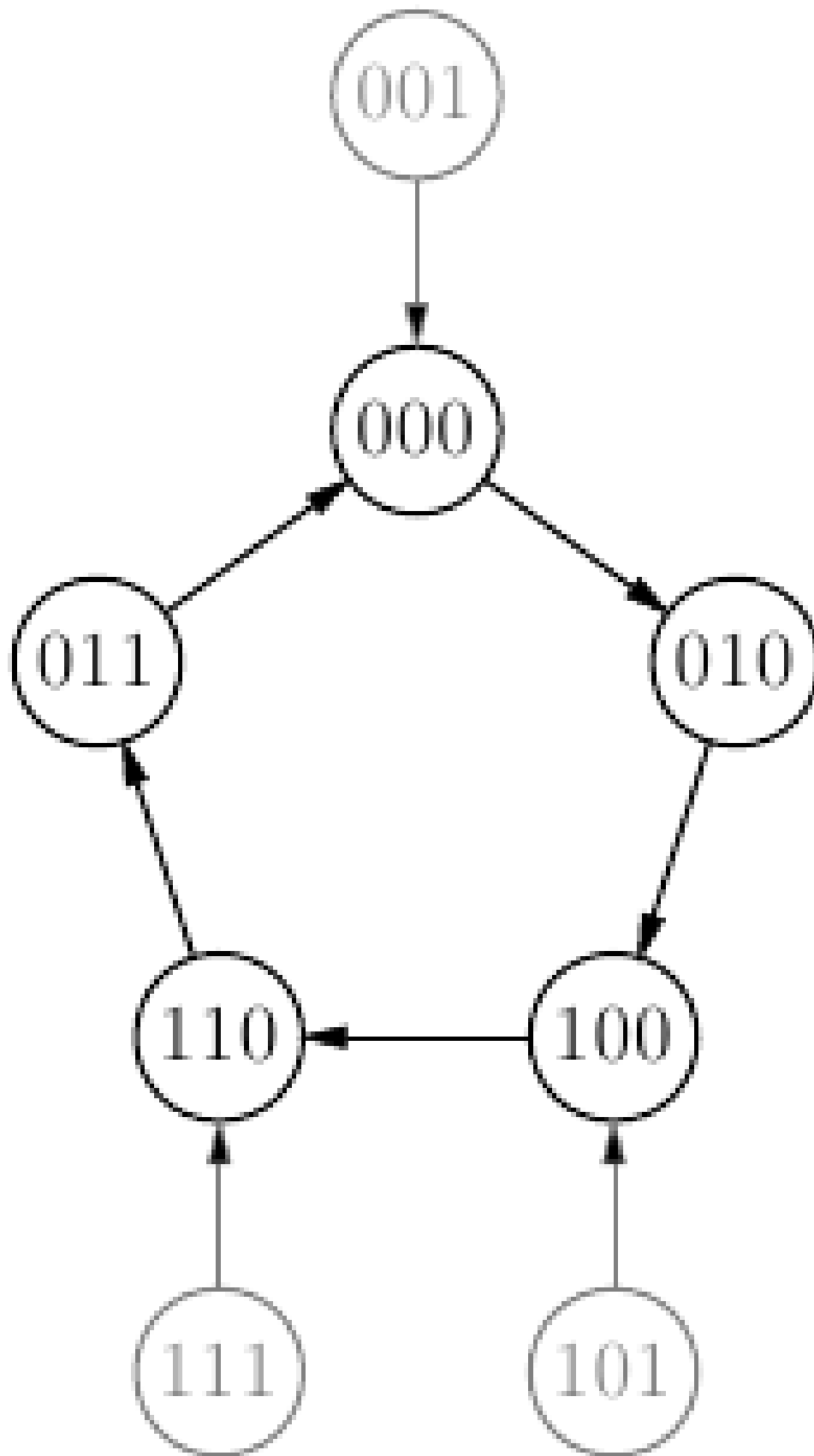


Figure8.5: Auxiliary counter circuit.



| Actual | Sgte. | J | K |
|--------|-------|---|---|
| 0      | 0     | 0 | x |
| 0      | 1     | 1 | x |
| 1      | 0     | x | 1 |
| 1      | 1     | x | 0 |

**Figure8.6:** Transitions of a JK flip-flop.

| Transición | J <sub>2</sub> | K <sub>2</sub> | J <sub>1</sub> | K <sub>1</sub> | J <sub>0</sub> | K <sub>0</sub> |
|------------|----------------|----------------|----------------|----------------|----------------|----------------|
| 000 → 010  | 0              | x              | 1              | x              | 0              | x              |
| 001 → 000  | 0              | x              | 0              | x              | x              | 1              |
| 010 → 100  | 1              | x              | x              | 1              | 0              | x              |
| 011 → 000  | 0              | x              | x              | 1              | x              | 1              |
| 100 → 110  | x              | 0              | 1              | x              | 0              | x              |
| 101 → 100  | x              | 0              | 0              | x              | x              | 1              |
| 110 → 011  | x              | 1              | x              | 0              | 1              | x              |
| 111 → 110  | x              | 0              | x              | 0              | x              | 1              |

**Figure8.7:** State transition diagram.

The Fig. 8.8 is the reference for: karnaugh map for JK inputs.

|                      |  | $J_2$ |   | $\leftarrow Q_2$ |
|----------------------|--|-------|---|------------------|
| $Q_1 Q_0 \downarrow$ |  | 0     | 1 |                  |
| 00                   |  | 0     | x |                  |
| 01                   |  | 0     | x |                  |
| 11                   |  | 0     | x |                  |
| 10                   |  | 1     | x |                  |

$$J_2 = Q_1 \overline{Q_0}$$

|                      |  | $K_2$ |   | $\leftarrow Q_2$ |
|----------------------|--|-------|---|------------------|
| $Q_1 Q_0 \downarrow$ |  | 0     | 1 |                  |
| 00                   |  | x     | 0 |                  |
| 01                   |  | x     | 0 |                  |
| 11                   |  | x     | 0 |                  |
| 10                   |  | x     | 1 |                  |

$$K_2 = Q_1 \overline{Q_0}$$

Figure8.8: Karnaugh map for JK inputs.

The Fig. 8.9 is the reference for: arbitrary-count counter.

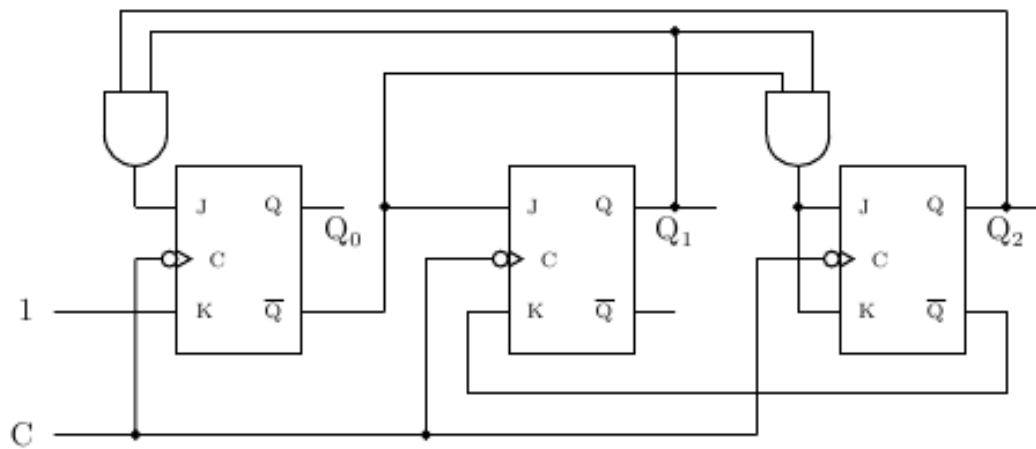


Figure8.9: Arbitrary-count counter.

## 8.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

## 8.6 Original source

Content translated and expanded from the class presentation and the reference page: <http://avellano.fis.usal.es/~compi/sesion8.htm>.



## SESSION 9. ALU

The final session applies modular work to a 74181 ALU. It practices file inclusion and the combination of arithmetic and logic operations.

### Learning objectives

- `include` inserts the contents of another Verilog file at the indicated point.
- The 74181 ALU receives selection lines that determine the operation.
- Negated carry lines require care when interpreting inputs and outputs.

## 9.1 Key concepts

- `include` inserts the contents of another Verilog file at the indicated point.
- The 74181 ALU receives selection lines that determine the operation.
- Negated carry lines require care when interpreting inputs and outputs.
- Compound operations are solved by storing intermediate results.
- The test module must clearly state the inputs, selected operation, and expected output.

## 9.2 Guided practice

1. Include the `74181.v` file from a test module; identify the ports in [Fig. 9.1](#).
2. Check operations such as  $7+4$ ,  $2+6+1$ , AND, XOR, and OR combined with NOT/XNOR; select the control lines using [Fig. 9.2](#).
3. Implement small multiplications as repeated additions; use [Fig. 9.2](#) again to justify each intermediate step.
4. Document each control-line selection so that errors can be debugged; your table must explicitly refer to [Fig. 9.1](#) and [Fig. 9.2](#).

## 9.3 Theory-practice-figures index

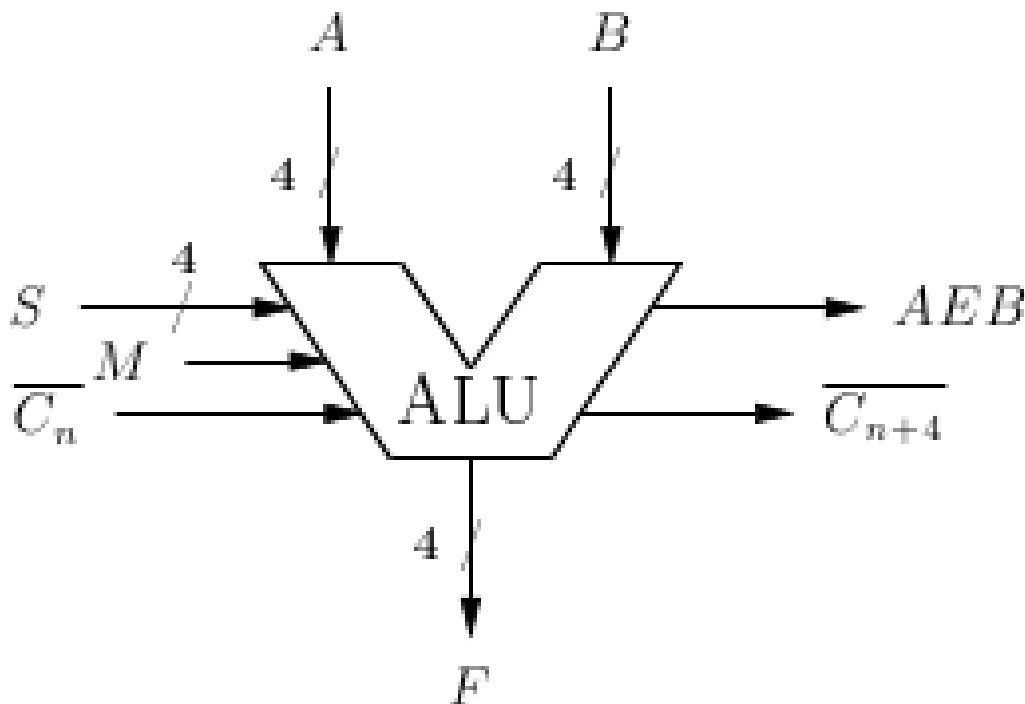
Use this table as a quick map: when an exercise mentions a figure, that figure is part of the statement and should be consulted before writing the code.

| Theory block        | Related exercises | Figures to consult       |
|---------------------|-------------------|--------------------------|
| ALU interface       | Exercise 1        | <a href="#">Fig. 9.1</a> |
| Operation selection | Exercises 2 to 4  | <a href="#">Fig. 9.2</a> |

## 9.4 Reference figures

The following figures collect the diagrams and tables that are useful while solving the exercises in this session.

The [Fig. 9.1](#) is the reference for: functional diagram of the 74181 ALU.



**Figure 9.1:** Functional diagram of the 74181 ALU.

The [Fig. 9.2](#) is the reference for: operation table of the 74181 ALU.

| $S$  | Aritmético ( $M = 0$ )                   | Lógico ( $M = 1$ )          |
|------|------------------------------------------|-----------------------------|
| 0000 | $A + C_n$                                | $\overline{A}$              |
| 0001 | $A \downarrow B + C_n$                   | $\overline{A \downarrow B}$ |
| 0010 | $A \downarrow \overline{B} + C_n$        | $\overline{A}B$             |
| 0011 | $-1 + C_n$                               | 0                           |
| 0100 | $A + A\overline{B} + C_n$                | $\overline{A}B$             |
| 0101 | $(A \downarrow B) + A\overline{B} + C_n$ | $\overline{B}$              |
| 0110 | $A - B - 1 + C_n$                        | $A \oplus B$                |
| 0111 | $A\overline{B} - 1 + C_n$                | $A\overline{B}$             |
| 1000 | $A + AB + C_n$                           | $\overline{A} \downarrow B$ |
| 1001 | $A + B + C_n$                            | $\overline{A \oplus B}$     |
| 1010 | $(A \downarrow \overline{B}) + AB + C_n$ | $B$                         |
| 1011 | $AB - 1 + C_n$                           | $AB$                        |
| 1100 | $A + A + C_n$                            | 1                           |
| 1101 | $(A \downarrow B) + A + C_n$             | $A \downarrow \overline{B}$ |
| 1110 | $(A \downarrow \overline{B}) + A + C_n$  | $A \downarrow B$            |
| 1111 | $A - 1 + C_n$                            | $A$                         |

**Figure9.2:** Operation table of the 74181 ALU.

## 9.5 Closing checkpoint

Before moving to the next session, save each Verilog exercise file and write down two things: what you expected to obtain and what the simulation actually printed. That comparison is the fastest way to locate errors.

## 9.6 Original source

Content translated and expanded from the class presentation and the reference page: <http://avellano.fis.usal.es/~compi/sesion9.htm>.